Freight Rate Prediction - Validation & Split Report

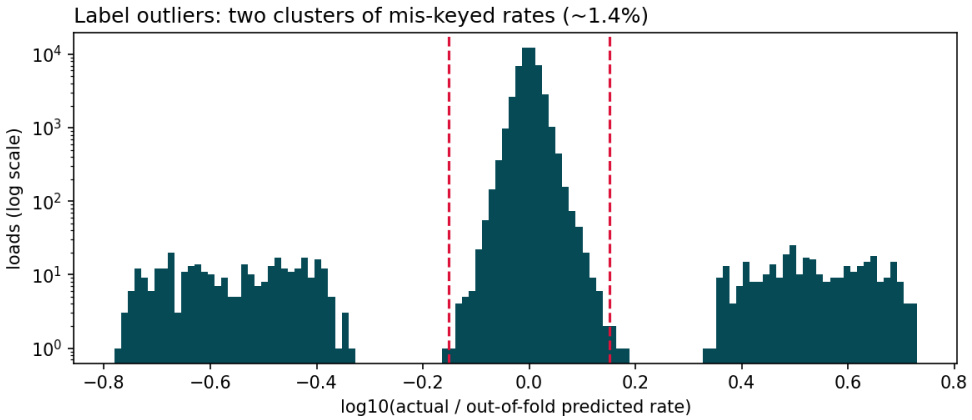
Model: LightGBM on load-level features + linear daily market/calendar effect. Target: log(posted_rate). Training data 2025-01-01..2025-10-31 (48,000 loads); prediction set 2025-11-01..2025-12-31 (12,000 loads).

1. What the split has to reproduce

The validation loads are entirely **later in time** than the training loads (Nov-Dec vs Jan-Oct) and **12.1% of them involve one of 8 cities** that never appear in training. A random K-fold split would therefore over-state accuracy twice: it lets the model interpolate between neighbouring days, and it never asks it to price an unseen city. The split was designed to reproduce both conditions.

2. Validating the train/test data before splitting

Check	Finding	Handling
Mis-keyed posted_rate	682 rows (1.42%), two clusters at ~0.29x and ~3.4x the expected rate	Flagged with 5-fold out-of-fold residuals (|log residual| > 0.35); removed from training only
Negative weight	292 train / 145 valid	Sign error: residuals match |weight|, so use abs()
Missing weight	300 train / 165 valid	Left as NaN (LightGBM native) + missing flag
Weight = 47,500 lb	1,204 train / 301 valid	A cap, kept as a real value
Missing market_index	374 train / 249 valid	Filled with that day's mean (index is ~daily: within-day sd 0.025 vs 0.167 across days)
Unseen cities	8 cities, 1,447 valid rows	Cities enter via coordinates only, never as IDs
Market shift	mean market_index 1.08 train vs 0.93 valid	Market effect modelled as an explicit elasticity, not left to trees
Duplicates / bad coords / bad distance	none found	Coordinates constant per city; distance consistent per lane



Out-of-fold residual ratios. The bulk is a tight peak; the two isolated clusters are mis-keyed rates. Keeping them doubles backtest error (table below).

3. Split strategy

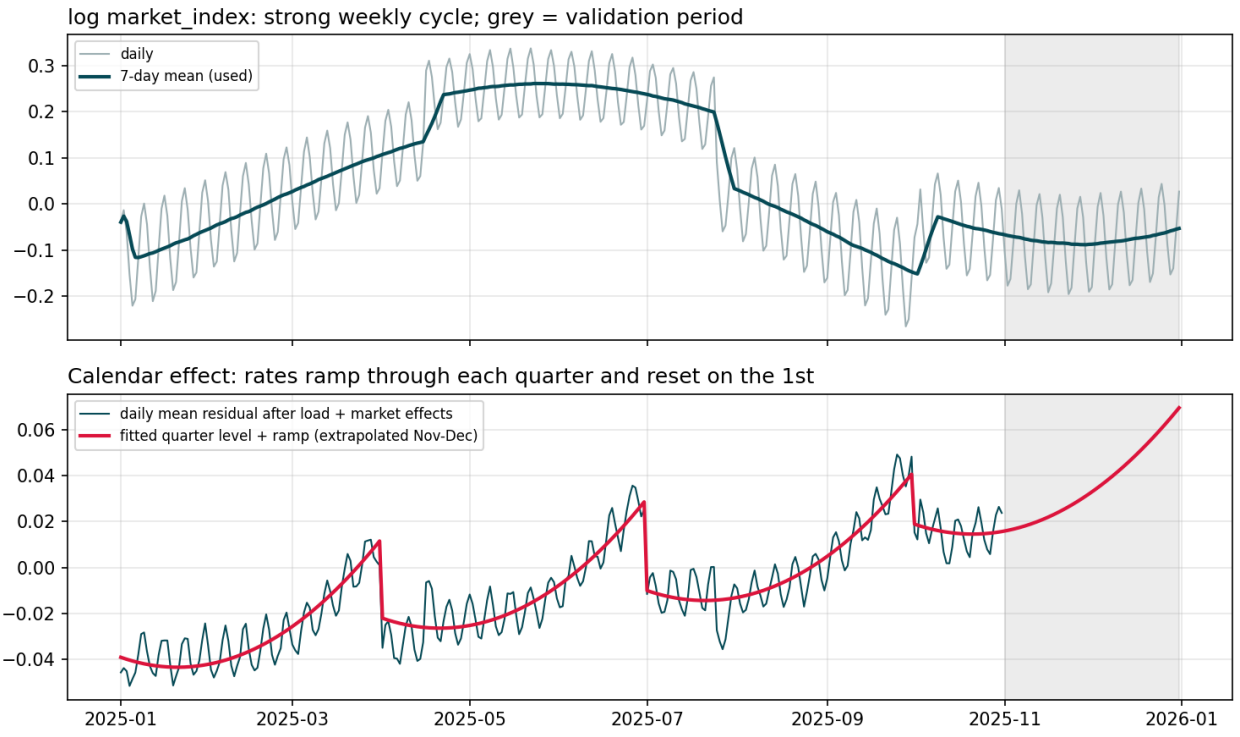
Rolling-origin time split. Each fold trains on everything before a cut-off and predicts the next two months, the same horizon as the real task. The real task observes October (the first month of Q4) and predicts November and December, so the two main folds have the same shape: *train through April, predict May-June*, and *train through July, predict August-September*. A third stress fold (train through August, predict September-October) also crosses a quarter boundary. All cleaning that uses labels (outlier filtering, smearing correction) is re-fitted inside each fold, so no

test labels leak into training. Test-period mis-keyed labels are reported separately because no model can predict a typo.

Cold-city split. 8 cities were removed from training entirely (every load touching them), and the model was scored on test loads that touch them. The cities drawn include a whole Texas/New Mexico cluster (Austin, Dallas, Amarillo, Lubbock, Albuquerque), which is harder than the real validation set, whose new cities are spread out.

4. Why the model is shaped this way

After controlling for lane, equipment and weight, rates follow the **7-day mean** of log market_index (elasticity about 0.15). The same-day value fits worse, because rates do not follow the index's strong weekly cycle. A clear **calendar pattern** also remains: rates climb through every quarter and reset on Apr 1, Jul 1 and Oct 1, and each quarter starts higher than the last. A tree model cannot extrapolate this into Nov-Dec. Worse, when trees were given day-level market features, they used them as a proxy for the calendar, and error on the later folds doubled. The final model therefore puts everything that is constant within a day into a small linear model: beta x 7-day market index + quarter level + convex within-quarter ramp. LightGBM models load-level structure: distance, coordinates, equipment, weight, and load-level deviations of market_index and quote_signal. The two parts are fitted by backfitting. The Q4 level is identified from October.



5. Backtest results (clean test labels)

Model	Apr known: May-Jun	Jul known: Aug-Sep	Stress: Sep-Oct	Mean MAPE %	Mean MAE $	MAPE % (incl. bad labels)
Final: GBM + daily effect	1.90	2.05	2.14	**2.03**	47.1	4.37
Same, no calendar terms	2.17	4.56	5.25	**3.99**	94.1	6.24
Final, outliers kept	4.39	4.09	3.58	**4.02**	92.3	6.35
Baseline rate-card	6.17	3.67	3.68	**4.50**	102.2	6.79

MAPE = mean absolute % error on rate. The calendar terms matter most exactly where the validation set sits (2nd-3rd month of a quarter). The 'incl. bad labels' column is dominated by the ~1.4% unpredictable mis-keyed test labels.

Cold-city check (stress fold, 8 cities hidden from training):

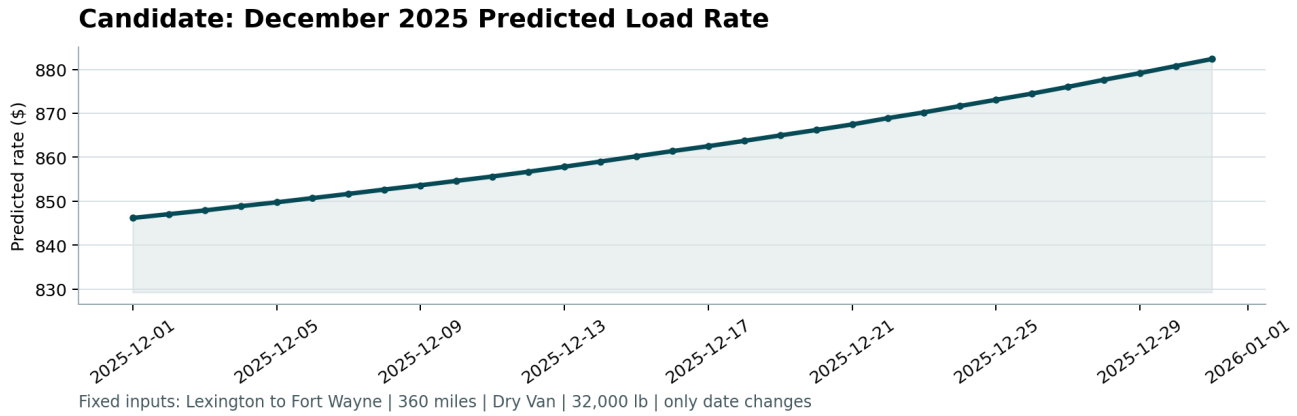
subset	n	MAE	RMSE	MAPE_%	MedAPE_%
seen_cities	7660	47.09	68.15	2.09	1.76
unseen_cities	1717	76.71	111.80	3.19	2.48

Unseen cities cost roughly one extra point of MAPE in this deliberately harsh setup, and the model still beats the rate-card baseline on the same fold (3.19% vs 3.68% MAPE).

6. Final fit

The final model is re-fitted on all 48,000 training loads, using the same cleaning and the same outlier rule. It predicts all 12,000 validation loads, and every value is positive and finite.

7. Fixed December prediction chart (produced by score.py)



Lexington to Fort Wayne, 360 mi, Dry Van, 32,000 lb. The December input rows have no market_index or quote_signal, so they get that day's market-wide values from validation.csv. The curve rises about 4% through the month (about $846 to $882), which is the learned quarter-end ramp. It is smooth because rates follow the 7-day market mean, not the daily cycle. For reference, comparable Dry Van loads on this lane in training were priced about $760-$935. Main assumption: the within-quarter ramp learned from Q1-Q3 repeats in Q4, whose level is identified from October alone, so late-December rates carry the most uncertainty.